PROJECT REPORT · DEPARTMENT OF DATA SCIENCE AND ENGINEERING, MANIPAL UNIVERSITY JAIPUR

MedTwin

Realistic and private synthetic patient data, including the values that were never measured

A model that creates realistic fake patient records from real hospital data. We tested it on two public medical datasets against three widely used methods, and built new ways to check whether the *missing* parts of the fake data are realistic and whether they leak private information.

Pranav Ladha · Devisha Jain · Nandini Nahata · Jiya Gakhar

October 2026

61.5%

how often a classifier can spot our fake heart patients (50% = impossible to spot; other methods 74.9–92.8%)

0.98

usefulness of our fake data compared with real data (1.00 = just as useful)

0.17

missing-value pattern error (two real samples: 0.15; other methods 0.21–0.68)

Contents

1 Summary

2 The problem in simple words

3 Key ideas explained

4 The data

5 How MedTwin works

6 Technical specifications

7 How we test it

8 Results

9 Comparison with previous work

10 What is new in this project

11 Limitations

12 Future work

13 Project history

A How to run the project

B References

1 Summary

MedTwin is a program that learns from real patient records and then makes new, fake patient records that look and behave like the real ones. Nobody in the fake data is a real person, so researchers can use it to build and test medical AI without seeing private patient files.

The hard part is that real hospital data has holes, and different hospitals measure different things. In our heart data, *no* Swiss patient has a cholesterol value and almost no Hungarian patient has a vessel-count result. Most existing tools fill these holes with guesses before learning. Their fake data then has no holes at all, which looks wrong and is easy to spot.

MedTwin is a *conditional Wasserstein GAN*. For every value that can be missing, it also learns a yes/no "was this measured?" flag. It also creates each patient *for a specific hospital*, so it picks up each hospital's habits. On top of the model, we built new tests that check whether the holes in fake data are realistic and whether keeping them puts privacy at risk.

Each experiment was run with five different random seeds, and testing always used patients kept away from training. A classifier trying to pick out our fake heart patients was right 61.5% of the time; a coin flip would score 50%. For the other three methods the figure was 74.9%, 79.5% and 92.8%. A disease predictor trained only on our fake data worked 97.5% as well as one trained on real data for heart disease, and 98.3% as well for diabetes. That is level with the best other method.

If you compare which values are missing, our fake patients differ from the real ones only a little more than two groups of real patients differ from each other, and no other method got the holes into the right hospitals. We attacked the fake data to look for copied patients and found none for MedTwin, while TabDDPM did copy on the diabetes data. There is one catch. Keeping realistic holes makes patients with *unusual* holes slightly easier to identify (attack score 0.575 against 0.531 without the flags).

2 The problem in simple words

Medical AI needs data that is hard to share

To build a computer program that predicts, for example, heart disease, we need many real patient records. These records are private. Hospitals keep them locked away, and privacy laws (HIPAA in the USA, GDPR in Europe) back that up. So students, researchers and companies often cannot get the data they need.

The idea: synthetic data

Synthetic data means fake records produced by a computer model that has learned the patterns in real records. A good synthetic patient looks real (an older patient tends to have a lower maximum heart rate, and a sick patient tends to have chest pain), but does not belong to any real person. If the fake data is good enough, people can share it instead of the real data.

People usually judge a synthetic dataset on the three questions below.

1. Realistic	2. Useful	3. Private
Does it look like real data, column by column and in how columns relate?	Does a model trained on the fake data work on real patients?	Is it impossible to work out which real patients were used to make it?

The twist: real data has holes

Medical tables are full of empty cells, and where they appear is not random. A test may be skipped because of hospital policy, the doctor's choice or the patient's condition. Three examples from our heart data:

- in Switzerland cholesterol is stored as 0 for everyone, a code for “no test”;
- the vessel count from a scan is empty for 96–99% of patients at three of the four hospitals;
- 28% of VA Long Beach patients have no resting blood pressure, while the other hospitals almost always record it.

Most synthetic-data tools cannot handle holes, so people fill them with a typical value (for example the median) first. What comes out is a table with every cell filled, which real hospital data never is, and the filled-in values come from no actual measurement. This led to three questions:

> **Project questions**
>
> 1. Can we generate realistic, useful synthetic patients *including* their holes, hospital by hospital?
> 2. How do we *measure* whether the holes in fake data are realistic?
> 3. Does keeping realistic holes create a new *privacy risk*?

3 Key ideas explained

These terms appear throughout the report.

Term	Meaning in simple words
Generative AI / generator	A model that learns from examples and then produces new examples of the same kind.
GAN (Generative Adversarial Network)	Two networks that compete: a *generator* makes fake records, a *critic* tries to tell fake from real. Both improve until the fakes are hard to spot [15].
Wasserstein GAN with gradient penalty (WGAN-GP)	A more stable kind of GAN. The critic gives a "realness score" instead of a yes/no answer, and a penalty keeps training smooth [16, 17].
Conditioning	Telling the generator what kind of record to make. We tell it the hospital, so it learns each hospital's habits.
Missing-value flag	An extra yes/no value per column: "was this measured?" The generator learns these too, and leaves a value blank when it says "no".
Held-out patients	20% of real patients that no model ever sees during training. We test on them so the results are fair.
Seed	A number that fixes the random choices. We repeat everything with 5 seeds to check results are not luck.
TSTR	Train on Synthetic, Test on Real: train a disease predictor on fake data and test it on real held-out patients [20].
AUC	A score for how well a predictor separates two groups: 1.0 = perfect, 0.5 = coin flip.
Detectability	How often a classifier can tell a fake record from a real one. 50% = impossible to tell (best).
Membership inference attack (MIA)	An attack that tries to find out whether a given real patient was used to train the generator. AUC 0.5 = attack fails (good) [21, 22].
DCR (distance to closest record)	Checks whether fake records sit suspiciously close to training patients. 50% = no sign of copying.
Ablation	Switching off one part of the system to see how much that part matters.
Paired 95% confidence interval	A statistical test across the 5 seeds. We call a difference "clear" only when the interval does not include zero.

4 The data

We use two public medical datasets. Both are small, which is typical of medical research and makes the task harder.

	Heart disease (UCI)	Pima diabetes
Patients	920	768
Columns used	7 numbers + 7 categories + hospital	9 numbers
Hospitals	4: Cleveland, Hungary, Switzerland, VA Long Beach	1 (one clinic)
Prediction target	has heart disease (severity > 0): 55.3% yes	has diabetes: 34.9% yes
Missing values	many, depending strongly on hospital	zeros that mean "not measured"
Source	UCI Machine Learning Repository [1]; collected by Detrano et al. [13]	OpenML dataset 37 [2]; Smith et al. [14]

Heart disease: the columns

Column	Meaning	Type	Missing
`age`	Age in years	number	0.0%
`sex`	Sex	category	0.0%
`dataset`	Hospital (site)	site	0.0%
`cp`	Chest pain type	category	0.0%
`trestbps`	Resting blood pressure (mmHg)	number	6.5%
`chol`	Serum cholesterol (mg/dl)	number	22.0%
`fbs`	Fasting blood sugar > 120 mg/dl	category	9.8%
`restecg`	Resting ECG result	category	0.2%
`thalch`	Maximum heart rate (bpm)	number	6.0%
`exang`	Chest pain during exercise	category	6.0%
`oldpeak`	ST depression on ECG	number	6.7%
`slope`	Slope of ST segment	category	33.6%
`ca`	Major vessels seen on scan (0–3)	number	66.4%
`thal`	Thalassemia test result	category	52.8%
`num`	Disease severity (0 = none, 1–4)	number	0.0%

Heart disease: missing values depend on the hospital

Share of patients with the value *not measured*, per hospital (number of patients in brackets). This pattern is exactly what a realistic synthetic dataset must reproduce.

Hospital	chol	trestbps	ca	thal	slope	fbs
Cleveland (304)	0%	0%	2%	1%	0%	0%
Hungary (293)	8%	0%	99%	90%	65%	3%
Switzerland (123)	100%	2%	96%	42%	14%	61%
VA Long Beach (200)	28%	28%	99%	83%	51%	4%

Pima diabetes: the columns

768 women of Pima heritage, aged 21 or older. Nobody alive has a glucose, blood pressure, skin fold, insulin or BMI of 0, so a 0 in those columns stands for a missing test. A 0 for pregnancies, on the other hand, is a real answer.

Column	Meaning	Missing
`preg`	Pregnancies	0.0%
`plas`	Glucose (mg/dL)	0.7% (0 = not measured)
`pres`	Diastolic BP (mmHg)	4.6% (0 = not measured)
`skin`	Skin fold (mm)	29.6% (0 = not measured)
`insu`	Insulin (mu U/ml)	48.7% (0 = not measured)
`mass`	BMI	1.4% (0 = not measured)
`pedi`	Diabetes pedigree	0.0%
`age`	Age (years)	0.0%
`outcome`	Diabetes (1 = yes)	0.0%

5 How MedTwin works

The whole system is one pipeline. Each step is a separate, tested part of the code.

1 Load settings
A YAML file describes the dataset: columns, rules, which zeros mean missing.

2 Split
80% of patients for training, 20% held out for testing (never seen).

3 Encode
Mark missing values, add yes/no flags, scale numbers to 0–1, one-hot categories.

4 Train
Conditional WGAN-GP learns real patients, hospital by hospital (600 epochs).

5 Generate
Create 2000 fake patients, each for a hospital drawn in real proportions.

6 Decode
Back to real units; leave a value blank when its flag says "not measured".

7 Clean
Medical range check and rounding to how hospitals record values.

8 Evaluate
Realism, usefulness, privacy and missing-value tests on held-out patients.

Step by step

The code itself knows nothing about heart disease or diabetes. A short settings file (for example `configs/heart_disease.yaml`) tells it the column types, the hospital column, the zero codes for "not measured" and the allowed range of each value. To add the diabetes data we only wrote a second settings file.

The main idea is how MedTwin handles missing values. For every number column that is sometimes missing, it adds a yes/no *measured flag*. The generator learns these flags together with the values, so it learns *which* values tend to be missing and *where*. When it creates a patient and the flag says "not measured", the value is left blank. A category column simply gets an extra option for "not measured".

In the heart data, 40% of ST depression values are exactly 0. A neural network cannot easily produce one exact number, so MedTwin learns a separate yes/no "is it exactly zero?" flag, which we call the zero flag.

The hospital is not generated. It is *given* to the generator, which is called hospital conditioning. We pick a hospital, MedTwin makes a patient for it, and the critic checks that patient against real patients *from the same hospital*. This is how site-specific habits (like no cholesterol test in Switzerland) are learned.

The generator also pays a penalty, the correlation loss, whenever the way its columns move together drifts away from the real data. This keeps relationships such as "older → lower maximum heart rate" in place.

Last, each value is rounded to the precision hospitals use (ST depression to 0.1, for example), and any patient outside the medical limits is dropped. The limits are wide enough that no real patient would fail them.

6 Technical specifications

Model architecture

Part	Specification
Generator input	random noise (128 numbers) + one-hot hospital
Generator layers	Linear 256 → LeakyReLU(0.2) → BatchNorm → Linear 512 → LeakyReLU → BatchNorm → Linear (output)
Generator outputs	Sigmoid for numbers and flags (0–1); Gumbel-softmax (τ = 0.2) per category, exactly one-hot when sampling
Critic layers	Linear 512 → LeakyReLU → Dropout 0.3 → Linear 256 → LeakyReLU → Dropout 0.3 → Linear 1 (no sigmoid)
Critic input	record + hospital, so it judges realism per hospital
Loss	Wasserstein loss [16] + gradient penalty (λ = 10) [17] + correlation penalty (α = 2)

Training settings

Setting	Value
Optimiser	Adam [18], learning rate 0.0001, betas (0.0, 0.9) for both networks
Epochs / batch size	600 epochs, batch size 64
Critic updates	5 critic updates per generator update
Records generated	2000 per run
Train / test split	80% / 20%, stratified by outcome; 5 seeds (0–4) change both the split and the model start
Training time	about 7 min (heart) and 4 min (diabetes) per run on a laptop CPU, several runs in parallel

Methods compared

Method	What it is	Training time (heart, avg.)
CTGAN [3]	Conditional tabular GAN, 300 epochs, batch 100	1.2 min
TVAE [3]	Variational autoencoder for tables, 300 epochs, batch 100	0.4 min
TabDDPM [4]	Diffusion model for tables (our compact reimplementation), 8000 steps	3.2 min
MedTwin (ours)	Conditional WGAN-GP with missing-value flags, 600 epochs	6.8 min

Software

Area	Tools and versions
Language	Python 3.9
Deep learning	PyTorch 2.8.0 [27]
Data	pandas 2.3.3, NumPy 2.0.2
Machine learning / tests	scikit-learn 1.6.1 [28] (random forests [19], nearest neighbours, AUC)
Baselines	ctgan 0.12.1 (CTGAN, TVAE); TabDDPM written in this project
Statistics	SciPy (paired t-tests)
Charts	matplotlib 3.9.4, seaborn 0.13.2
Dashboard	Streamlit 1.50.0
Settings	PyYAML 6.0.3
Testing	pytest (31 automated tests)

Code structure

File / folder	Purpose
`main.py`	Runs the full pipeline once and saves the synthetic data and scores
`core/config.py`	Loads a dataset's settings file
`core/ingestion.py`	Missing-value handling, flags, scaling, encoding
`core/generator.py`	Generator, critic and WGAN-GP training
`core/medtwin.py`	Train-and-generate steps; ablation switches
`core/medical_logic.py`	Medical range check and rounding
`core/evaluation.py`	Usefulness, privacy and correlation tests on held-out patients
`core/missingness.py`	New missing-value realism scores and privacy attacks
`research/baselines.py`, `tabddpm.py`	The three comparison methods
`research/multi_seed.py`	5-seed comparison with paired statistics
`research/ablation.py`	Ablation and sensitivity studies
`research/visualizer.py`, `paper_tables.py`	Figures and paper tables
`app/dashboard.py`	Interactive Streamlit dashboard
`configs/*.yaml`	One settings file per dataset
`tests/`	31 automated tests
`paper/`	Research paper (Springer LNCS format)

Every experiment can be rerun from the code. After we added new measurements we ran everything again, and all 400 saved results matched exactly.

7 How we test it

A fair test setup

One patient in five (20%) is put aside before training, and no model ever sees them. These are the patients we use to measure usefulness and privacy. Testing on patients the model had seen would make the results look better than they are. Everything is repeated with 5 seeds. Each seed holds out different patients and starts the models differently, and we report the average and the spread (± standard deviation).

All four methods get the same data, missing-value flags, cleaning and tests. Methods are compared seed by seed, and we call a difference *clear* only if its 95% confidence interval does not include zero.

What we measure

Measure	Question it answers	Best value	Group
Fidelity	Do single columns look real? (overlap of histograms)	higher, max 100%	realism
Detectability	Can a classifier tell fake from real?	50%	realism
Correlation gap	Do relationships between columns match?	0	realism
TSTR AUC	How good is a disease predictor trained on fake data, tested on real patients?	higher	usefulness
AUC ratio	TSTR AUC divided by the same predictor trained on real data	1.00	usefulness
DCR	Are fake records closer to training patients than to unseen ones?	50%	privacy
MIA AUC	Can an attacker tell which patients were used for training?	0.5	privacy
Missing-rate gap new	Is each column missing as often as in real data?	0	missing values
Site gap new	Is each column missing as often *at each hospital*?	0	missing values
Pattern distance new	Do whole patterns of holes (which values are missing together) match?	0	missing values
Rare-pattern MIA new	Are patients with unusual holes easier to identify?	0.5	privacy of holes
Unique-pattern leak new	Do one-of-a-kind hole patterns of training patients reappear?	0	privacy of holes

> **Detectability is not privacy**
>
> Some papers call the "can a classifier spot the fakes?" score a privacy score. That is a mistake. A model that just copied real patients would be impossible to spot and would still leak everything. We use it only for realism, and judge privacy with DCR and membership attacks.

The new missing-value measures, in simple words

We call the set of blank values in a patient's record, together with the hospital, their *missingness pattern*. An example is "Hungary: cholesterol measured, vessel scan missing, thalassemia missing".

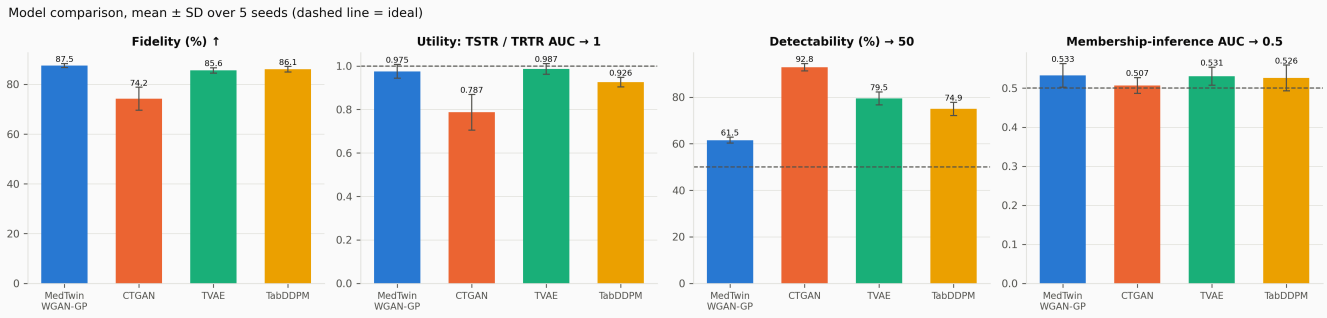
- The pattern distance compares how often each pattern appears in fake and in real data (0 = identical, 1 = completely different). Because the score depends on sample size, we also compute it between two groups of *real* patients. A fake dataset that scores close to this real-vs-real value is as good as more real data.
- The rare-pattern attack aims the membership attack at patients whose pattern is shared by 2% of patients or fewer, and compares the result with the rest. If keeping holes hurts privacy, rare patients suffer most.

8 Results

All numbers are averages over 5 seeds ± standard deviation, measured on real patients the models never saw. In the headers, ↑ means higher is better, ↓ lower is better, and → means the closer to that value the better. Best values in bold are described in the text.

8.1 Heart disease: MedTwin against the other methods

Method	Fidelity % ↑	Detect. % →50	TSTR AUC ↑	AUC ratio →1	DCR % →50	MIA AUC →0.5
MedTwin (ours)	87.5 ± 0.8	61.5 ± 1.3	0.863 ± 0.023	0.975 ± 0.032	52.9 ± 2.8	0.533 ± 0.030
TVAE	85.6 ± 1.1	79.5 ± 2.8	0.873 ± 0.022	0.987 ± 0.024	50.1 ± 3.4	0.531 ± 0.023
TabDDPM	86.1 ± 1.1	74.9 ± 2.8	0.820 ± 0.030	0.926 ± 0.022	52.7 ± 3.0	0.526 ± 0.033
CTGAN	74.2 ± 4.6	92.8 ± 1.6	0.696 ± 0.069	0.787 ± 0.082	51.1 ± 3.4	0.507 ± 0.020



Heart disease comparison, averages over 5 seeds with spread. Dashed lines show the ideal value.

- MedTwin has the best fidelity (87.5%) and much the lowest detectability (61.5%, against 74.9–92.8%). Both differences are clear against every method.
- For usefulness, TVAE (0.987) and MedTwin (0.975) are statistically level, and MedTwin is clearly ahead of CTGAN (0.787).
- DCR and MIA are close to the ideal for all methods, and no method copied a real patient.

8.2 Diabetes: the same picture

Method	Fidelity % ↑	Detect. % →50	TSTR AUC ↑	AUC ratio →1	DCR % →50	MIA AUC →0.5
MedTwin (ours)	90.9 ± 0.8	64.0 ± 2.6	0.801 ± 0.030	0.983 ± 0.041	51.8 ± 1.8	0.494 ± 0.021
TVAE	88.0 ± 1.7	70.0 ± 1.4	0.806 ± 0.013	0.989 ± 0.028	54.0 ± 2.4	0.518 ± 0.024
TabDDPM	85.4 ± 1.3	55.9 ± 2.6	0.781 ± 0.050	0.957 ± 0.026	60.8 ± 4.9	0.803 ± 0.028
CTGAN	77.8 ± 3.3	86.3 ± 2.7	0.568 ± 0.062	0.697 ± 0.075	49.7 ± 7.7	0.505 ± 0.028

- Fidelity is again highest for MedTwin (90.9%), and its usefulness score (0.983) is level with the best.
- TabDDPM is the hardest to spot (55.9%) but it memorised its training patients. 60.8% of its records sit closer to training patients and the membership attack reaches AUC 0.803, matching recent findings that diffusion models for tables memorise [24]. MedTwin shows no such leak (0.494). So a low detectability score tells you nothing about privacy.

8.3 Are the missing values realistic?

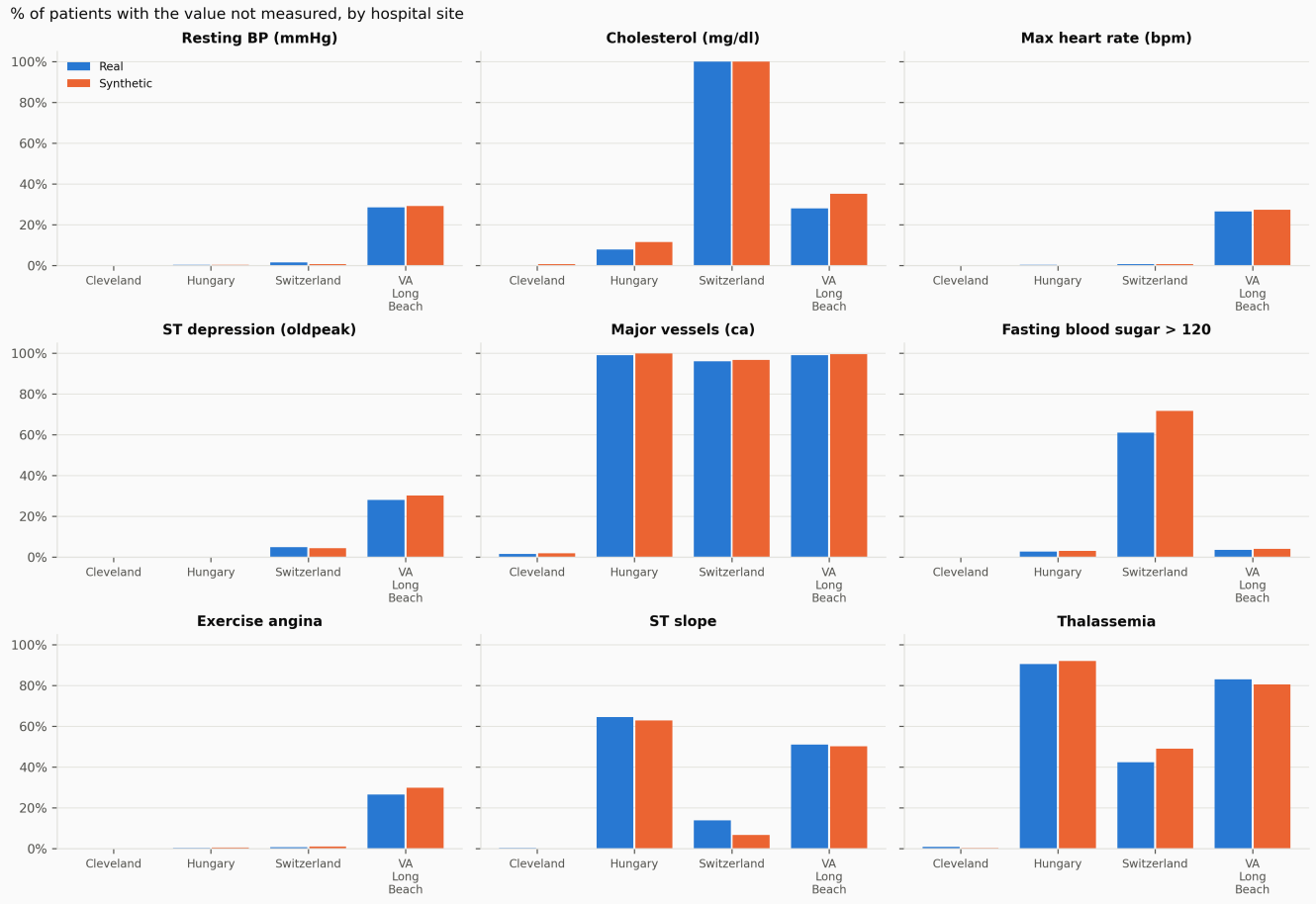
Heart disease

	Rate gap ↓	Site gap ↓	Pattern dist. ↓	MIA rare →0.5	MIA common →0.5
Two real samples (reference)	0.020	0.024	0.152		
MedTwin (ours)	0.021 ± 0.002	0.022 ± 0.001	0.174 ± 0.006	0.559 ± 0.037	0.545 ± 0.037
TVAE	0.028 ± 0.005	0.035 ± 0.002	0.209 ± 0.009	0.566 ± 0.060	0.541 ± 0.031
TabDDPM	0.032 ± 0.005	0.101 ± 0.004	0.487 ± 0.028	0.501 ± 0.032	0.539 ± 0.037
CTGAN	0.059 ± 0.005	0.131 ± 0.006	0.683 ± 0.014	0.544 ± 0.070	0.515 ± 0.029

Diabetes (one clinic, so there is no site gap)

	Rate gap ↓	Site gap ↓	Pattern dist. ↓	MIA rare →0.5	MIA common →0.5
Two real samples (reference)	0.030	–	0.085		
MedTwin (ours)	0.019 ± 0.002	–	0.063 ± 0.006	0.501 ± 0.177	0.506 ± 0.017
TVAE	0.031 ± 0.005	–	0.118 ± 0.022	0.549 ± 0.152	0.518 ± 0.029
TabDDPM	0.029 ± 0.010	–	0.111 ± 0.013	0.682 ± 0.081	0.799 ± 0.028
CTGAN	0.089 ± 0.033	–	0.299 ± 0.032	0.501 ± 0.202	0.509 ± 0.023

- Two real samples differ by a pattern distance of 0.152, and MedTwin's fake data reaches 0.174. It beats each other method clearly in all 5 seeds (TVAE 0.209, TabDDPM 0.487, CTGAN 0.683).
- Only MedTwin gets the hospital right: its site gap (0.022) matches real data (0.024). The other methods had the same flags, yet nothing tells them which hospital a hole belongs to, and the holes end up at the wrong sites.
- Taking the flags away pushes the pattern distance up to 0.693; taking away hospital conditioning raises it to 0.223 and the site gap to 0.028.



How often each heart disease value is missing at each hospital: real data in blue, MedTwin in orange.

8.4 Does keeping missing values cost privacy?

Yes, a little, for one group. Each heart disease test attacks 368 patients, and roughly 64 of them have a *rare* missingness pattern. Against these patients the attack scores an AUC of **0.575** on MedTwin, but only 0.531 if MedTwin is trained without the missing-value flags. The gap shows up in all 5 seeds. For patients with common patterns (0.542 vs 0.516) and overall (0.532 vs 0.515) there is no clear difference. A pattern that only one training patient has shows up in the fake data no more often than a pattern from an unseen patient, so the model does not copy patterns. The diabetes data has only about 8 rare-pattern patients per test, which is not enough to say anything.

> **What this means**
>
> Keeping the holes makes the fake data look more real and does not change privacy for the average patient. Patients whose holes are *unusual*, though, get a little easier to single out. Data owners should know this. Merging rare patterns before release is one fix worth testing.

8.5 What each part contributes (ablation)

Heart disease (* = clear difference from the full model)

Variant	Detect. % →50	Pattern dist. ↓	Corr. gap ↓	AUC ratio →1	MIA AUC →0.5
Full MedTwin	62.2 ± 1.7	0.171 ± 0.006	0.041 ± 0.007	0.974 ± 0.028	0.532 ± 0.030
without missing-value flags	84.2 ± 1.2*	0.693 ± 0.009*	0.071 ± 0.007*	0.989 ± 0.025	0.515 ± 0.027
without zero flag	74.9 ± 0.9*	–	–	0.978 ± 0.032	0.539 ± 0.038
without hospital conditioning	65.6 ± 1.7*	0.223 ± 0.012*	0.046 ± 0.005	0.975 ± 0.038	0.527 ± 0.025
without correlation loss	65.2 ± 2.4	–	0.075 ± 0.018*	0.982 ± 0.011	0.525 ± 0.032

Diabetes

Variant	Detect. % →50	Pattern dist. ↓	Corr. gap ↓	AUC ratio →1	MIA AUC →0.5
Full MedTwin	64.0 ± 2.6	0.063 ± 0.006	0.034 ± 0.007	0.983 ± 0.041	0.494 ± 0.021
without missing-value flags	81.5 ± 1.6*	0.486 ± 0.018*	0.042 ± 0.004*	0.993 ± 0.032	0.524 ± 0.055
without correlation loss	63.8 ± 2.9	–	0.063 ± 0.008*	0.964 ± 0.032	0.508 ± 0.029

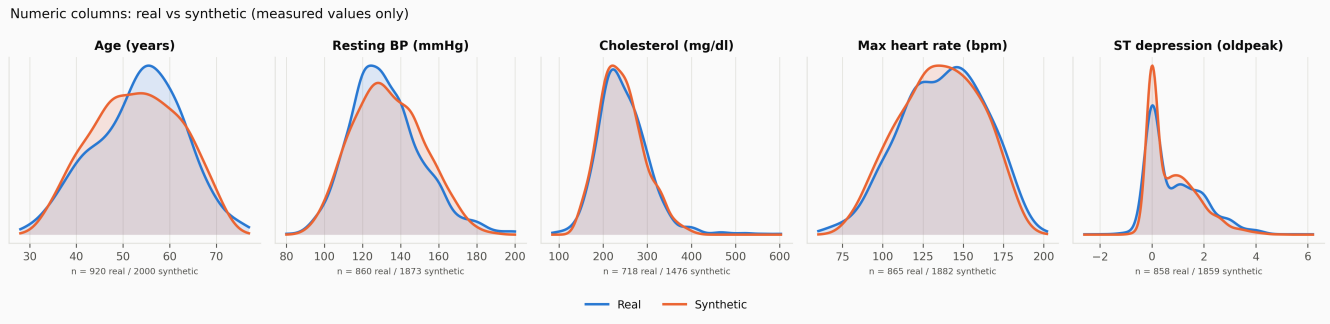
- The missing-value flags matter most. Remove them and detectability goes from 62.2% to 84.2% on heart disease, and from 64.0% up to 81.5% on diabetes. Filling holes with typical values makes fake patients easy to spot.
- Dropping the zero flag or hospital conditioning also makes the records clearly easier to spot (74.9% and 65.6%).
- Realism and usefulness stay the same without the correlation loss, but the correlation gap almost doubles (0.041 → 0.075 heart; 0.034 → 0.063 diabetes).
- No part changes usefulness or overall privacy. Not a single patient failed the medical range check, so it only guards against rare bad outputs.

8.6 Do results depend on our chosen settings?

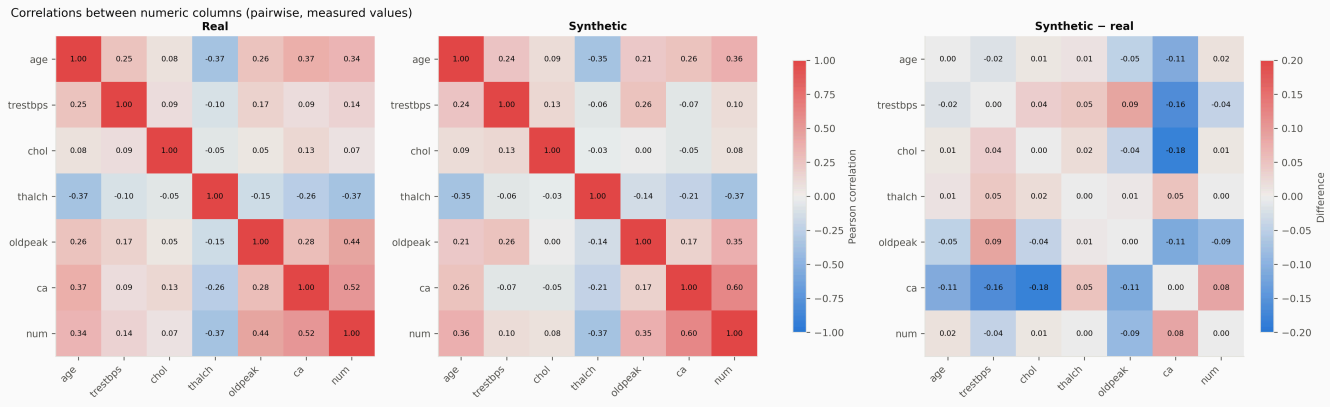
Setting (heart disease)	Fidelity %	Detect. %	AUC ratio	DCR %	MIA AUC
Default (weight 2, 600 epochs)	87.2 ± 0.7	62.2 ± 1.7	0.974 ± 0.028	53.2 ± 2.0	0.532 ± 0.030
Correlation weight 0.5	87.0 ± 0.8	62.8 ± 2.8	0.983 ± 0.021	53.3 ± 2.1	0.531 ± 0.023
Correlation weight 8	87.9 ± 0.5	62.8 ± 2.3	0.989 ± 0.031	53.9 ± 2.6	0.525 ± 0.032
300 epochs	87.0 ± 0.5	63.6 ± 2.8	0.961 ± 0.034	52.2 ± 2.5	0.524 ± 0.035
1200 epochs	87.2 ± 0.5	63.0 ± 2.2	0.984 ± 0.017	55.3 ± 2.6*	0.548 ± 0.024*

A correlation weight of 0.5 or 8 (instead of 2), or a shorter run of 300 epochs, gives no clear difference. Training twice as long (1200 epochs) starts to memorise: DCR and MIA rise clearly. We kept 600 epochs, and the main results hold across these settings.

8.7 A closer look at the generated values



Heart disease number columns, measured values only: real patients in blue, MedTwin in orange.



Correlations between number columns: real, MedTwin, and the difference (heart disease).

9 Comparison with previous work

We searched for work from the last five years on the same datasets or the same problem. Since every paper picks its own data, test split and scores, their figures and ours are not directly comparable. The table shows what each did and how our project relates. Larger benchmark studies of synthetic health data [25, 26] compare many generators on realism, usefulness and privacy, and we judge our model the same way.

Work	What they did	Their result	How we compare
Wan Abu Bakar et al., 2025 (IJACSA) [5]	CTGAN on Cleveland heart data (303 patients); missing values filled with most common value; single run; GAN saw all patients	Accuracy train-on-fake, test-on-real: 56% (5-class)	We use all 4 hospitals, unseen test patients and 5 seeds. Our CTGAN baseline is also weak; MedTwin clearly beats it.
Subedi et al., 2025 (IEEE ICUIS) [6]	CTGAN on Pima diabetes	Random forest accuracy 82% (real) → 63% (fake), i.e. 77% kept	MedTwin keeps 98% of real-data performance (AUC ratio, 5 seeds). Setups differ, so this is indicative.
Alqulaity & Yang, 2024 (Sensors) [7]	Enhanced conditional GAN, 218 cardiovascular patients, one hospital	KS statistic 0.390 (TVAE 0.386); no usefulness or privacy test	Our evaluation covers realism, usefulness and privacy with unseen patients and repeats.
Lazzaro et al., 2026 (J. Comp. Science) [8]	Bayesian network on 3 UCI heart sites, trained on Cleveland only	Augmentation results	We model all 4 hospitals including the very incomplete ones.
HealthGen, 2022 [9]; MissDiff, 2023 [10]; Wang et al., 2025 (IEEE TKDE) [11]; IGAMT, 2024 (AAAI) [12]	Generators that keep missing values (time series, diffusion, plug-in, private EHR)	Show that keeping missingness matters	Same goal. Our additions: hospital-conditioned holes, standard scores for hole realism, and the first test of the privacy cost of holes. We have not yet compared against them directly.

On the same datasets, our results and our testing go further than recent published work. Other groups were first to keep missing values in synthetic data. What we add is a way to *measure* whether the holes are realistic, hospital conditioning, and evidence of what the holes cost in privacy.

10 What is new in this project

1. We score how realistic the missing values are: the missing rate per column and per hospital, and the full pattern of which values go missing together. Each score is shown next to the gap between two real samples, so the reader can see what a good value looks like. Existing papers have no common measure for this.
2. We test whether realistic holes make patients easier to identify. As far as we know, nobody had checked this before. Patients with rare patterns pay a small cost that standard privacy tests do not show.
3. MedTwin is the only tested method that puts holes at the right hospital, because each patient is generated for a given hospital.
4. The evaluation uses unseen test patients, five repeats and paired statistics. It also switches off each part of the model in turn, checks the main settings, and keeps realism and privacy apart.

11 Limitations

- Both datasets have fewer than 1000 patients and one table each. Larger hospital databases and data collected over time may behave differently.
- A classifier can still spot our fake heart patients 62% of the time (50% would be perfect).
- We tested privacy with attacks. There is no mathematical guarantee such as differential privacy, and passing these attacks does not prove the data is safe [23].
- The missing-value privacy audit needs enough patients with rare patterns. It works on heart disease but not on diabetes.
- The newest missing-value generators (MissDiff, Wang et al.) and TabSyn were not run in our benchmark, and TabDDPM is our own compact version.

12 Future work

1. Compare directly with MissDiff and Wang et al., the closest previous work.
2. Test fixes for the privacy cost, such as merging rare missing-value patterns before release.
3. Add formal privacy (differentially private training) and measure what it costs in realism.
4. Train on three hospitals and generate patients for the fourth, which would help hospitals share data.
5. Move to larger and time-based hospital data.

13 Project history

The first version, from April 2026, was much smaller: 9 columns, gaps filled in, and tests on patients the model had trained on. We then rebuilt it to keep the gaps in all 15 columns, condition on the hospital, test on held-out patients and compare with other methods over five seeds. The recorded steps since then:

Date	Step
2026-09-29	Snapshot before 15-column, missing-data and conditional-generation work
2026-09-29	Generate all 15 columns with 'not measured' handling and site conditioning
2026-09-29	Oldpeak zero flag, 15-column 5-seed comparison, paper figures
2026-09-29	Experiment: move dataset-specific settings into configs/heart_disease.yaml
2026-09-29	Experiment: run the pipeline on Pima diabetes from configs/diabetes.yaml
2026-10-15	Add ablation switches and a multi-seed ablation runner
2026-10-15	Rename privacy audit to detectability, add sensitivity study, draft related work
2026-10-15	Add ablation, sensitivity and diabetes 5-seed results; fix mixed-name loading
2026-10-15	Add correlation-gap metric, rerun correlation ablation, write LNCS paper
2026-10-15	Add missing-value fidelity scores and privacy audit; refocus paper on them
2026-10-15	Add project report PDF and the script that builds it
2026-10-16	Complete and verify all citations in the paper and project report
2026-10-16	Rewrite paper sentences that followed their sources too closely
2026-10-16	Shorten abstract and add Springer-required statements to the paper
2026-10-16	Build the paper PDF and fix its layout
2026-10-16	Update the author list in the paper and project report
2026-10-16	Add MUJ-format project report and the scripts that build it
2026-10-16	Fix spelling of Nandini Nahata in the paper and project report
2026-10-17	Restyle the MUJ report to match the department template exactly
2026-10-17	Match the MUJ report's spacing and table text to the template
2026-10-19	Rewrite the paper and project reports in plainer, more natural English
2026-10-19	Cut the MUJ report to its 15 most relevant references and add summaries
2026-10-19	Cut the MUJ report to its 15 most relevant references and add summaries

A How to run the project

From the project folder (Windows):

Task	Command
Install	`python -m venv venv && venv\Scripts\pip install -r requirements.txt`
Run the pipeline (heart)	`python main.py`
Run on diabetes	`python main.py --config configs/diabetes.yaml`
Dashboard	`streamlit run app/dashboard.py`
5-seed comparison	`python research/multi_seed.py`
Ablations	`python research/ablation.py`
Settings check	`python research/ablation.py --study sensitivity`
Figures	`python research/visualizer.py --benchmark`
Paper tables	`python research/paper_tables.py`
Tests	`python -m pytest tests -q`
This report	`python docs/build_report.py`

A full 5-seed comparison takes about an hour on a laptop when seeds run in parallel. Results are saved per run, so an interrupted run continues where it stopped.

B References

1. Janosi, A., Steinbrunn, W., Pfisterer, M., Detrano, R.: Heart Disease [dataset]. UCI Machine Learning Repository (1989). https://doi.org/10.24432/C52P4X
2. Vanschoren, J., van Rijn, J.N., Bischl, B., Torgo, L.: OpenML: networked science in machine learning. ACM SIGKDD Explorations Newsletter 15(2), 49–60 (2014). https://doi.org/10.1145/2641190.2641198 (Pima diabetes: OpenML dataset 37)
3. Xu, L., Skoularidou, M., Cuesta-Infante, A., Veeramachaneni, K.: Modeling tabular data using conditional GAN. In: Advances in Neural Information Processing Systems 32 (NeurIPS) (2019)
4. Kotelnikov, A., Baranchuk, D., Rubachev, I., Babenko, A.: TabDDPM: Modelling tabular data with diffusion models. In: Proc. 40th International Conference on Machine Learning, PMLR 202, 17564–17579 (2023)
5. Wan Abu Bakar, W.A., Josdi, N.L.N., Man, M., Kadir, E.A.: Evaluating CTGAN-generated synthetic data for heart disease prediction: fidelity, predictive utility, and feature preservation. International Journal of Advanced Computer Science and Applications 16(12), 1000–1009 (2025). https://doi.org/10.14569/IJACSA.2025.0161296
6. Subedi, P., Divakarla, S.R., Emani, M., Kochuvila, S., Sunitha, R.: Synthetic data for health risk prediction: an evaluation using machine learning. In: 5th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS), IEEE (2025). https://doi.org/10.1109/ICUIS67429.2025.11380688
7. Alqulaity, M., Yang, P.: Enhanced conditional GAN for high-quality synthetic tabular data generation in mobile-based cardiovascular healthcare. Sensors 24(23), 7673 (2024). https://doi.org/10.3390/s24237673
8. Lazzaro, I., Milano, M., Tucker, A., Cannataro, M.: Assessing the impact of synthetic data generated by Bayesian networks on heart disease prediction. Journal of Computational Science 99, 102940 (2026). https://doi.org/10.1016/j.jocs.2026.102940
9. Bing, S., Dittadi, A., Bauer, S., Schwab, P.: Conditional generation of medical time series for extrapolation to underrepresented populations (HealthGen). arXiv:2201.08186 (2022)
10. Ouyang, Y., Xie, L., Li, C., Cheng, G.: MissDiff: Training diffusion models on tabular data with missing values. ICML 2023 Workshop on Structured Probabilistic Inference and Generative Modeling. arXiv:2307.00467 (2023)

11. Wang, X., Asif, H., Gupta, S., Vaidya, J.: Data synthesis reinvented: preserving missing patterns for enhanced analysis. IEEE Transactions on Knowledge and Data Engineering 37(7), 3962–3975 (2025)
12. Wang, W., Tang, P., Lou, J., Shao, Y., Waller, L., Ko, Y., Xiong, L.: IGAMT: Privacy-preserving electronic health record synthesization with heterogeneity and irregularity. In: Proc. AAAI Conference on Artificial Intelligence 38(14), 15634–15643 (2024). https://doi.org/10.1609/aaai.v38i14.29491
13. Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer, M., Schmid, J.J., Sandhu, S., Guppy, K.H., Lee, S., Froelicher, V.: International application of a new probability algorithm for the diagnosis of coronary artery disease. The American Journal of Cardiology 64(5), 304–310 (1989). https://doi.org/10.1016/0002-9149(89)90524-9
14. Smith, J.W., Everhart, J.E., Dickson, W.C., Knowler, W.C., Johannes, R.S.: Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. In: Proc. Annual Symposium on Computer Application in Medical Care, 261–265 (1988)
15. Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. In: Advances in Neural Information Processing Systems 27, 2672–2680 (2014)
16. Arjovsky, M., Chintala, S., Bottou, L.: Wasserstein generative adversarial networks. In: Proc. 34th International Conference on Machine Learning, PMLR 70, 214–223 (2017)
17. Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., Courville, A.: Improved training of Wasserstein GANs. In: Advances in Neural Information Processing Systems 30 (NeurIPS) (2017)
18. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: 3rd International Conference on Learning Representations (ICLR) (2015)
19. Breiman, L.: Random forests. Machine Learning 45(1), 5–32 (2001). https://doi.org/10.1023/A:1010933404324
20. Esteban, C., Hyland, S.L., Rätsch, G.: Real-valued (medical) time series generation with recurrent conditional GANs. arXiv:1706.02633 (2017)
21. Hayes, J., Melis, L., Danezis, G., De Cristofaro, E.: LOGAN: Membership inference attacks against generative models. Proceedings on Privacy Enhancing Technologies 2019(1), 133–152 (2019). https://doi.org/10.2478/popets-2019-0008
22. Chen, D., Yu, N., Zhang, Y., Fritz, M.: GAN-Leaks: A taxonomy of membership inference attacks against generative models. In: Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS) (2020). https://doi.org/10.1145/3372297.3417238
23. Stadler, T., Oprisanu, B., Troncoso, C.: Synthetic data – anonymisation groundhog day. In: 31st USENIX Security Symposium, 1451–1468 (2022)
24. Fang, Z., Jiang, Z., Chen, H., Li, X., Li, J.: Understanding and mitigating memorization in diffusion models for tabular data. In: Proc. 42nd International Conference on Machine Learning, PMLR 267, 15976–16005 (2025)
25. Yan, C., Yan, Y., Wan, Z., Zhang, Z., Omberg, L., Guinney, J., Mooney, S.D., Malin, B.A.: A multifaceted benchmarking of synthetic electronic health record generation models. Nature Communications 13, 7609 (2022). https://doi.org/10.1038/s41467-022-35295-1
26. Hernandez, M., Osorio-Marulanda, P.A., Catalina, M., Loinaz, L., Epelde, G., Aginako, N.: Comprehensive evaluation framework for synthetic tabular data in health. Frontiers in Digital Health 7, 1576290 (2025). https://doi.org/10.3389/fdgth.2025.1576290
27. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., et al.: PyTorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems 32 (NeurIPS) (2019)
28. Pedregosa, F., Varoquaux, G., Gramfort, A., et al.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12, 2825–2830 (2011)